

```

6.     if isinstance(msg, str):
7.         msg = msg.encode("utf-8")
8.     h = hashlib.sha256(msg).hexdigest()
9.     return Integer(int(h, 16) % n)

10. def sign_message(msg):
11.     h = hash_message(msg)
12.     sig = pow(h, d, n)
13.     return sig

14. def verify_signature(msg, sig):
15.     h = hash_message(msg)
16.     recovered = pow(sig, e, n)
17.     return recovered == h

18. message = "Hello Bob"
19. print("Message:", message)

20. signature = sign_message(message)
21. print("\nSignature:", signature)

22. ok = verify_signature(message, signature)
23. print("Verification:", ok)

24. tampered = message + "!"
25. print("Tampered message:", tampered)
26. ok2 = verify_signature(tampered, signature)
27. print("Verification on tampered message:", ok2)
    
```

Now, let us try to understand the working of the program. Line 1 imports Python's *hashlib* module so that we can compute SHA-256 hashes of messages. Lines 2 to 4 set the RSA modulus n (a public parameter), which is the product of two large prime numbers, the RSA public exponent e (commonly 65537), and the RSA private exponent d (which is kept secret). The value in variable d satisfies $e \cdot d \equiv 1 \pmod{\phi(n)}$.

In Line 5, the function *hash_message()* that returns an integer hash for a message begins. Line 6 checks whether the value in the variable *msg* is a Python string so that it can be encoded to bytes. If *msg* is text, Line 7 encodes it to UTF-8 bytes (required by *hashlib*). Line 8 computes the SHA-256 digest of the byte string *msg* and returns it as a hexadecimal string. Line 9 converts the hex digest *h* to an integer (base 16), reduces its modulo n to ensure the hash fits the RSA modulus, and returns it as a SageMath Integer.

In Line 10, the function *sign_message()*—which creates an RSA signature for *msg*—begins. Line 11 computes the integer hash *h* of the message using the function *hash_message()*. Line 12 computes the RSA signature $\text{sig} = h^d \bmod n$ using fast modular exponentiation. Line 13 returns the signature integer. In Line 14, the function *verify_signature()*—which checks whether *sig* is a valid

signature for *msg*—begins. Line 15 recomputes the integer hash *h* of the (received) message. Line 16 recovers the signed hash by computing $\text{sig}^e \bmod n$ (the inverse operation of signing). Line 17 returns *True* if the recovered value equals the computed hash *h*, otherwise *False* — this is the signature verification test.

Line 18 defines the message to be signed, which in this case is “Hello Bob”. Line 19 prints the message to the console. Line 20 calls the function *sign_message()* to produce the signature for the message and stores it in the variable *signature*. Line 21 prints the signature (the integer) to the console. Line 22 verifies the signature against the original message, and the result is stored in the Boolean variable *ok*. Line 23 prints the verification result (*True* if valid). Line 24 creates a tampered version of the message by appending an exclamation mark. Line 25 prints the tampered message. Line 26 attempts to verify the original signature against the tampered message and stores the result in the Boolean variable *ok2*. Finally, Line 27 prints the verification result for the tampered message.

Upon executing the program ‘*digsign41.sage*’, the output shown in Figure 2 is obtained. The first line displays the digital signature of the message, which in this case is 4466491779663873112179864100592158183363638. The second line indicates that the original message is authentic, and therefore the verification result is *True*. The figure also shows the modified message “Hello Bob!” (with an exclamation mark appended) fails verification, correctly producing the result *False*, since the message has been tampered with.

```

Signature: 4466491779663873112179864100592158183363638
Verification: True
Tampered message: Hello Bob!
Verification on tampered message: False
    
```

Figure 2: Output of a simple digital signature scheme

Message authentication with hashing

Now, it is time to implement message authentication using hashing in SageMath. As mentioned earlier, message authentication efficiently ensures message integrity. It relies on symmetric-key cryptography, in which both the sender and receiver share a common secret key. Because symmetric-key operations are used instead of public-key cryptography, message authentication is generally much faster than digital signatures. However, message authentication cannot provide non-repudiation, and regarding sender authentication, it only ensures that the message was sent by someone who possesses the shared secret key.

Now, let us implement a simplified version of a message authentication scheme using symmetric-key encryption and SHA-256. The SageMath program titled ‘*mac42.sage*’ (line numbers included for clarity),